CUSTOMER FEEDBACK INTELLIGENCE PLATFORM

Agentic AI Project Blueprint and Antigravity Implementation Specification

Primary goal Build a resume-ready, production-minded GenAI application through controlled phases and as few Antigravity prompts as practical.

Application type: AI-powered customer feedback decision-support platform

Domain: Banking feedback initially; domain-neutral architecture

Initial input: CSV ingestion

Architecture: Angular + Spring Boot + Python FastAPI/LangGraph + PostgreSQL/PGVector

Document purpose: Source of truth for planning, generation, testing, and handover

Version 1.0 | 26 October 2026

1. Executive Summary

The Customer Feedback Intelligence Platform converts large volumes of unstructured customer comments into structured, evidence-backed business insights. It is not merely a chatbot or sentiment dashboard. It combines reliable data ingestion, controlled AI processing, semantic retrieval, analytical tools, and agentic orchestration to help managers understand what happened, why it may be happening, and what should be investigated next.

Core architectural principle Use deterministic workflows for mandatory processing and agentic tool selection only for open-ended analytical questions. This balances autonomy, reliability, traceability, and testability.

1.1 Business problem

- Organizations receive feedback through reviews, surveys, email, support tickets, applications, and call transcripts.
- Manually reading and categorizing thousands of comments is slow, inconsistent, and difficult to scale.
- Leaders need quantified trends, themes, representative evidence, and practical next actions rather than raw comments alone.

1.2 Proposed solution

- Upload feedback through CSV in Version 1.
- Validate, deduplicate, and persist raw feedback as the source of truth.
- Use a controlled LangGraph workflow to classify feedback and create embeddings.
- Display KPIs, trends, categories, themes, priorities, and individual records.
- Allow managers to ask analytical questions through a tool-calling supervisor agent.
- Ground every answer in database metrics and retrieved feedback evidence.
- Generate cautious recommendations framed as investigation or action steps, not unverified root-cause claims.

1.3 Success definition

- A CSV file can be uploaded and its rows tracked from validation to completion or failure.
- AI outputs conform to a controlled schema and are traceable to model and prompt versions.
- Users can filter, inspect, keyword-search, and semantically search feedback.
- The agent can compare periods, retrieve evidence, and provide grounded answers with references.
- The full system runs locally with documented commands and contains no mock LLM in the final demonstration.

2. Scope, Users, and Product Boundaries

2.1 Primary users

User	Main need
Product/Operations Manager	Monitor customer experience and emerging issues.
Customer Support Manager	Understand recurring complaints and routing needs.
Department Lead	Review feedback relevant to a product or team.
Data/Platform Administrator	Upload datasets, monitor processing, investigate failures.

2.2 Version 1 features

- CSV upload with preview and column mapping
- File-level and row-level validation
- Batch status and progress monitoring
- Duplicate detection and failed-row reporting
- AI extraction for each feedback item
- Embedding generation and PGVector storage
- Executive dashboard
- Feedback explorer with filters
- Keyword and semantic search
- Agentic conversational analytics with evidence
- Retry/reprocess support
- Prompt/model traceability in stored analysis

2.3 Explicitly out of scope for initial release

- Live email, social-media, app-store, or support-ticket integrations
- Kafka, Kubernetes, Redis, and microservice-scale infrastructure
- Complex enterprise identity integration
- Fully autonomous free-form multi-agent conversations
- Automatic execution of operational changes
- Claims of proven technical root cause from feedback alone
- Cloud deployment before local acceptance criteria pass

2.4 Future enhancements

- Email, ticketing, survey, app-review, or REST ingestion adapters
- Role-based access control and SSO
- Multilingual translation and language-specific analysis
- Human review and classification correction workflow
- Scheduled insight generation and alerting
- Evaluation dashboard for prompts and models
- Cloud deployment, queues, observability, and autoscaling

3. High-Level Architecture

```
Angular Web Application
        | REST/JSON + multipart upload
        v
Spring Boot Business API
        | REST/JSON
        v
Python FastAPI AI Service
        | LangGraph workflows + tools
        +--------------------+
        |                    |
        v                    v
PostgreSQL + PGVector   Real LLM + Embedding Model
```

3.1 Responsibility boundaries

Layer	Responsibilities	Must not own
Angular	User experience, CSV selection/mapping, dashboards, explorer, chat, status polling.	Database access, prompts, API keys, AI orchestration.
Spring Boot	System API, validation, batches, raw persistence, status, dashboard/query endpoints, calls to AI service.	LLM reasoning and LangGraph state.
FastAPI/LangGraph	Analysis workflow, structured model output, embeddings, supervisor agent, analytical tools.	Frontend behavior and file-format business ownership.
PostgreSQL/PGVector	Relational source of truth, analysis, vectors, insights, failures, chat/tool audit data.	Application decisions.
Model provider	Language understanding, generation, embeddings.	Trusted calculations or direct unrestricted database access.

3.2 Key design rules

- Raw feedback is immutable and remains visible for audit.
- Numerical facts come from controlled SQL/tools, not model estimation.
- The model receives only the minimum relevant context.
- AI-generated output is schema-validated before persistence.
- Provider-specific code is isolated behind interfaces so the model can be changed later.
- Every processing state and error is observable.
- Secrets are loaded from environment variables and never committed.

3.3 Source adapter design

```
CSV Adapter ---------
Email Adapter (future) \
Ticket Adapter (future) ---> Canonical FeedbackRecord ---> Same processing pipeline
Review Adapter (future) /
```

4. Canonical Data and AI Output

4.1 Initial CSV fields

Field	Requirement	Purpose / example
feedback_text	Required	Original customer comment.
feedback_date	Required	When feedback was submitted.
channel	Required	mobile_app, web, support_ticket, survey.
external_reference	Optional	Source-system review or ticket identifier.
customer_reference	Optional	Non-sensitive pseudonymous identifier.
rating	Optional	Typically 1-5.
product	Optional	Mobile Banking, Cards, Transfers.
region	Optional	High-level geography only.
customer_segment	Optional	Retail, premium, business.
language	Optional	Detected if absent.

```
feedback_id,customer_id,feedback_text,rating,channel,feedback_date,product
F001,C101,"Login fails after entering OTP",1,mobile_app,2026-09-10,Mobile Banking
```

4.2 Required AI output per feedback

Attribute	Controlled examples	Notes
sentiment	POSITIVE, NEUTRAL, NEGATIVE	Single label.
category	AUTHENTICATION, TRANSFERS, PAYMENTS, CARDS, SUPPORT, MOBILE_APP, ACCOUNTS, OTHER	Stable business taxonomy.
theme	LOGIN_FAILURE, OTP_ISSUE, APP_CRASH, DUPLICATE_DEBIT, REFUND_DELAY, SLOW_PERFORMANCE, OTHER	Specific issue; taxonomy may evolve.
emotion	HAPPY, SATISFIED, CONFUSED, FRUSTRATED, DISAPPOINTED, ANGRY, NEUTRAL	Use only when reasonably supported.
priority	LOW, MEDIUM, HIGH, CRITICAL	Impact/urgency, not sentiment alone.
assigned_department	AUTH_ENGINEERING, PAYMENTS, MOBILE_ENGINEERING, CUSTOMER_SUPPORT, CARDS, ACCOUNTS, PRODUCT_OPERATIONS	Suggested routing.
short_summary	One factual sentence	No invented details.

confidence_score	0.00-1.00	Model self-confidence, not proof of correctness.

4.3 Safety and data-quality policy

- Do not ingest account numbers, card numbers, passwords, authentication secrets, or unnecessary personal identities.
- Mask or reject obvious sensitive patterns before sending text to the model.
- Treat model classification as assistive output; retain original text for verification.
- Recommendations must say “investigate,” “review,” or “compare” when the evidence cannot prove causation.

5. Database Blueprint

Use PostgreSQL for relational data and the PGVector extension for embeddings. Use UUID primary keys where practical, UTC timestamps, explicit foreign keys, indexes for frequent filters, and migration scripts rather than ad-hoc schema changes.

5.1 ingestion_batches

batch_id, file_name, source_type, uploaded_at, total_rows, valid_rows, successful_rows, failed_rows, status, file_hash, uploaded_by

5.2 feedback

feedback_id, batch_id, external_reference, customer_reference, feedback_text, rating, channel, product, region, customer_segment, language, feedback_date, received_at, processing_status, deduplication_hash

5.3 feedback_analysis

analysis_id, feedback_id, sentiment, category, theme, emotion, priority, assigned_department, short_summary, confidence_score, analyzed_at, model_name, prompt_version

5.4 feedback_embeddings

embedding_id, feedback_id, embedding vector, embedding_model, embedding_dimension, created_at

5.5 processing_errors

error_id, batch_id, feedback_id nullable, csv_row_number nullable, processing_stage, error_code, error_message, original_row JSON nullable, created_at, resolved

5.6 generated_insights - Phase 6+

insight_id, insight_type, title, summary, date_from, date_to, filters_used JSON, evidence JSON, generated_at, model_name, prompt_version

5.7 recommendations - Phase 6+

recommendation_id, insight_id nullable, recommended_action, target_department, priority, reason, status, generated_at

5.8 chat_sessions / chat_messages / tool_runs

session metadata, USER/ASSISTANT messages, tool name and sanitized inputs, tool result summary, retrieved feedback IDs, latency and error status

5.9 Relationships

```
ingestion_batches 1 ---- * feedback
feedback          1 ---- 0..* feedback_analysis
feedback          1 ---- 0..* feedback_embeddings
ingestion_batches 1 ---- * processing_errors
generated_insights 1 --- * recommendations
chat_sessions     1 ---- * chat_messages
chat_messages     1 ---- * tool_runs
```

Versioning decision Allow multiple analyses or embeddings per feedback instead of overwriting previous outputs. Mark the active/latest result through a version or current flag if needed.

6. LangGraph Workflow A: Feedback Processing

```
START -> load_feedback -> clean_and_validate -> analyze_with_llm
    -> validate_ai_output -> create_embedding -> persist_results -> END
                  \-> retry_once -> error_handler -> END
```

6.1 Shared graph state

```
feedback_id, batch_id, raw_text, cleaned_text, analysis_result,
embedding, current_step, retry_count, processing_status, error_details
```

6.2 Nodes

Node	Type	Behavior
load_feedback	Deterministic	Retrieve a specific unprocessed record.
clean_and_validate	Deterministic	Trim/normalize, check length/language, detect obvious sensitive content.
analyze_with_llm	LLM	Return only the required structured schema with controlled values.
validate_ai_output	Deterministic	Validate JSON/schema/enums/ranges and apply correction retry when needed.
create_embedding	Model call	Embed approved combined text representation.
persist_results	Deterministic	Insert analysis/vector atomically where practical; update status.
error_handler	Deterministic	Write processing_errors and mark FAILED with stage and retry details.

6.3 Processing statuses

```
UPLOADED -> VALIDATED -> QUEUED -> PROCESSING -> COMPLETED
                                \-> FAILED -> RETRYING -> COMPLETED/FAILED
```

6.4 Reliability rules

- Idempotency: re-calling the same processing request must not create uncontrolled duplicates.
- Retry transient model/network failures with bounded attempts and backoff.
- Do not retry permanently invalid rows without correction.
- Batch processing must continue even if individual rows fail.

- Persist stage-specific failure information and use small concurrency limits to respect model quotas and database capacity.

7. LangGraph Workflow B: Agentic Analytics

```
User question -> supervisor -> choose tool -> inspect result
                          ^           |
                          |--- need more evidence?
                                      | no
                                      v
                          grounded answer + evidence
```

7.1 Approved tools

Tool	Use	Guardrail
metrics_tool	Counts, percentages, grouped KPIs.	Parameterised approved queries only.
trend_comparison_tool	Compare periods and calculate changes.	Calculations in code/SQL, not LLM.
semantic_search_tool	Meaning-based retrieval through PGVector.	Filters and top-k limits.
feedback_detail_tool	Fetch original records supporting a finding.	Return only necessary fields.
insight_tool	Interpret verified metrics and retrieved evidence.	No unsupported factual claims.
recommendation_tool	Suggest investigation/action steps.	Must cite evidence; no automatic execution.

7.2 Example execution

1. Question: “Why are negative reviews increasing this week and what should we investigate?”
2. Metrics tool establishes that negative share changed from one verified value to another.
3. Trend tool identifies the categories/themes contributing most to the change.
4. Semantic search retrieves representative comments.
5. Feedback detail tool supplies originals and IDs.
6. Recommendation tool converts verified evidence into cautious next steps.
7. Final answer contains conclusion, numbers, evidence references, limitations, and recommendations.

7.3 Agent answer contract

- Direct answer in two or three sentences
- Verified metrics with selected date range
- Main contributing themes/categories
- Representative feedback references
- Recommended investigations/actions
- Explicit limitation when evidence is insufficient

Important Expose tool activity and evidence, not hidden chain-of-thought. Example: “Compared weekly sentiment; retrieved 12 relevant feedback records.”

8. User Interface and Navigation

Screen	Minimum capabilities
Upload CSV	Select file, preview rows, map columns, validate, submit.
Processing Monitor	Batch totals, progress, status, failed rows, retry, error download.
Executive Dashboard	Date/product/channel filters; KPI cards; sentiment/category/theme/priority trends; AI summary.
Feedback Explorer	Table, filters, keyword search, semantic search, record detail, original text.
Ask the Agent	Question input, grounded answer, evidence links, understandable tool activity.
Insights - later	Saved weekly/trend/executive insights with evidence.
Recommendations - later	Priority, target department, reason, lifecycle status.

8.1 Navigation

```
Overview
Feedback Explorer
Ask the Agent
Data Operations
  - Upload CSV
  - Processing Monitor
  - Error Records
AI Insights (later)
Recommendations (later)
```

8.2 UX expectations

- Responsive, clean business dashboard rather than flashy demo styling.
- Every chart responds to active filters.
- Loading, empty, partial, and error states are explicit.
- Original feedback is always accessible from AI-derived information.
- Critical status colors are accessible and never the sole indicator.
- Do not fabricate dashboard values when APIs return no data.

9. API Contract Outline

Final route names may be adjusted once, but the functional boundaries should remain stable. Generate OpenAPI documentation for Spring Boot and FastAPI.

9.1 Spring Boot public APIs

```
POST   /api/v1/batches/validate          multipart CSV; return preview, mappings, errors
POST   /api/v1/batches                   upload and create batch
GET    /api/v1/batches/{id}              batch progress and counts
GET    /api/v1/batches/{id}/errors       paginated errors
POST   /api/v1/batches/{id}/retry        retry eligible failures
GET    /api/v1/dashboard/overview        filtered KPIs
GET    /api/v1/dashboard/trends          filtered time series
GET    /api/v1/feedback                  paginated filters/keyword search
GET    /api/v1/feedback/{id}             raw + current analysis
POST   /api/v1/feedback/semantic-search  semantic query + filters
POST   /api/v1/chat/sessions             create chat session
POST   /api/v1/chat/sessions/{id}/messages  ask analytical question
```

9.2 FastAPI internal APIs

```
POST   /internal/v1/analyze-feedback     process one record or controlled batch
POST   /internal/v1/embed                create embedding when explicitly required
POST   /internal/v1/semantic-search      vector search service
POST   /internal/v1/agent/query          supervisor agent query
GET    /internal/v1/health               service health/readiness
```

9.3 Integration rules

- Use DTOs; do not expose database entities directly.
- Return consistent error objects with correlation ID, code, message, and field errors.
- Use pagination and upper bounds for record-heavy endpoints.
- Apply client timeouts and bounded retries between services.
- Keep an internal service token or local-only network boundary; add stronger service authentication in deployment phase.

10. Technology and Repository Structure

Area	Technology
Frontend	Angular, TypeScript, chart library
Business backend	Java 21, Spring Boot, Maven
AI service	Python, FastAPI, LangGraph, LangChain integrations, Pydantic
Database	PostgreSQL + PGVector
Models	Real chat model + real embedding model, preferably Azure OpenAI when available
Infrastructure	Docker Desktop and Docker Compose
Testing	JUnit/MockMvc or equivalent; pytest; Angular tests; Postman collection
Version control	Git and GitHub

10.1 Monorepo recommendation

```
customer-feedback-intelligence/
  frontend-angular/
  backend-spring/
  ai-service-python/
  database/
    migrations/
    seed/
  sample-data/
  docs/
  postman/
  docker-compose.yml
  .env.example
  .gitignore
  README.md
```

10.2 Configuration

```
DATABASE_URL / DB_HOST / DB_PORT / DB_NAME / DB_USER / DB_PASSWORD
AI_SERVICE_BASE_URL
MODEL_PROVIDER
AZURE_OPENAI_ENDPOINT / AZURE_OPENAI_API_KEY / deployment names
CHAT_MODEL / EMBEDDING_MODEL
PROMPT_VERSION
CORS_ALLOWED_ORIGINS
```

- Commit .env.example with placeholders only.
- Never commit .env, secrets, model keys, database volumes, build output, or real sensitive feedback.
- Pin major dependency versions and record setup commands in README.

11. Phase-Wise Implementation Plan

Prompt-saving strategy Give Antigravity this entire document once. For each phase, ask it to inspect existing files, implement only that phase, run tests/builds, and return changed files plus exact commands. Do not ask it to regenerate the whole repository after Phase 1.

Phase 0 - Discovery and environment verification

Tasks:

- Inspect installed Java, Maven, Node/npm, Angular CLI, Python, Docker, Git.
- Confirm real model provider and deployment/model names without exposing keys.
- Create repository and environment checklist; make no speculative substitutes.

Definition of done: Documented versions; Model access confirmed; No code architecture divergence.

Phase 1 - Repository skeleton and infrastructure

Tasks:

- Create monorepo folders, README, .gitignore, .env.example.
- Create Docker Compose for PostgreSQL with PGVector.
- Scaffold Spring Boot, FastAPI, and Angular apps.
- Add health endpoints and verify each application starts.

Definition of done: All services build/start; Database persistent volume works; Health checks pass.

Phase 2 - Database and CSV ingestion

Tasks:

- Create migrations for batches, feedback, errors, analysis, embeddings.
- Implement canonical CSV mapping, preview, validation, deduplication, persistence, batch progress.
- Build Upload and Processing Monitor screens.
- Provide sample banking-feedback CSV with synthetic, clearly labelled data.

Definition of done: Valid file persists rows; Invalid rows are reported; Duplicate behavior is tested; UI shows real batch status.

Phase 3 - Controlled AI analysis workflow

Tasks:

- Implement provider abstraction and real model configuration.
- Implement LangGraph Workflow A, Pydantic schema, enum validation, bounded retry, persistence, prompt/model versioning.
- Process small batches first; add retry/reprocess endpoint.

Definition of done: Structured output validates; Failures are observable; At least one real-model batch completes; No mock path used for demonstration.

Phase 4 - Dashboard and feedback explorer

Tasks:

- Create filtered KPI and trend queries.
- Implement overview cards/charts and feedback explorer/detail.
- Add keyword search, pagination, filter state, loading/error/empty states.

Definition of done: Metrics reconcile with database queries; Filters work consistently; Original and AI data are visible.

Phase 5 - Embeddings, PGVector, semantic search, RAG

Tasks:

- Create vectors with real embedding model.
- Add vector index when dataset size justifies it.
- Implement semantic search with metadata filters and top-k.
- Return feedback IDs/scores and expose semantic search in explorer.

Definition of done: Meaning-based queries return relevant records; Results are traceable; No hallucinated records.

Phase 6 - Agentic conversational analytics

Tasks:

- Implement supervisor graph and approved tools.
- Add metrics, trend, semantic-search, detail, insight, and recommendation tools.
- Persist sessions/messages/tool audit data as needed.
- Build Ask the Agent screen with evidence and source links.

Definition of done: Agent chooses appropriate tools; Numbers match SQL results; Answers reference evidence; Unsupported questions admit limitations.

Phase 7 - Hardening and portfolio packaging

Tasks:

- Add unit/integration/end-to-end smoke tests.
- Add logging, correlation IDs, validation, PII handling, CORS, rate/size limits.
- Create architecture diagram, API docs, demo script, screenshots, sample questions, troubleshooting guide.
- Optionally containerize application services and prepare cloud deployment plan.

Definition of done: Clean fresh-machine setup; No secrets in Git; All builds/tests pass; README supports recruiter demonstration.

12. Testing, Evaluation, and Non-Functional Requirements

12.1 Test levels

Level	Examples
Unit	CSV validators, dedup hash, status transitions, output schema, calculations, tool input validation.
Integration	Spring-PostgreSQL, FastAPI-model provider, PGVector retrieval, Spring-FastAPI contracts.
Contract	OpenAPI DTOs, error format, enum compatibility.
End-to-end	Upload sample CSV -> process -> dashboard -> semantic search -> agent question.
AI evaluation	Label accuracy sample, schema success rate, retrieval relevance, groundedness, recommendation safety.

12.2 Practical evaluation set

- Create a small reviewed set of 50-100 feedback records with expected sentiment/category/theme.
- Maintain 10-20 semantic-search queries with expected relevant records.
- Maintain 10 analytical questions with expected tool calls and verifiable numeric answers.
- Record model, prompt version, date, pass rate, and known failure patterns.

12.3 Non-functional requirements

- Clear status and recoverability are more important than maximum throughput for Version 1.
- Do not block a browser request while thousands of model calls complete; use background processing or a simple controlled worker pattern.
- All list endpoints are paginated.
- File type, row count, file size, and text length have configurable limits.
- Logs contain correlation IDs but not model keys or unnecessary sensitive text.
- System remains usable when some feedback rows fail.

13. Antigravity Operating Instructions

Use the following as the persistent instruction block at the start of the Antigravity conversation. Attach this blueprint if the tool supports attachments; otherwise paste the relevant phase plus these global rules.

```
You are implementing the Customer Feedback Intelligence Platform described in the attached blueprint.
Treat the blueprint as the product and architecture source of truth.

GLOBAL RULES
1. Work only on the requested phase. Do not pre-build later phases unless a tiny dependency is unavoidable.
2. Before editing, inspect the repository and summarize what already exists and what will change.
3. Preserve working code. Modify incrementally; do not regenerate completed applications.
4. Use Angular for UI, Spring Boot for business APIs, FastAPI/LangGraph for AI orchestration, and PostgreSQL/PGVector for storage.
5. Keep deterministic processing separate from agentic decision-making.
6. Do not create mock LLM behavior as the final implementation. If model credentials are missing, finish the provider interface/configuration and clearly stop at the credential-dependent verification step.
7. Never hard-code secrets. Update .env.example and README when configuration changes.
8. Use migrations, DTOs, validation, structured errors, pagination, and tests.
9. Run relevant builds/tests after implementation. Fix failures caused by your changes.
10. At the end, provide: summary, changed files, architecture decisions, commands to run, tests executed/results, remaining known issues, and the exact next-phase prompt.
11. Do not claim success unless commands/tests actually pass.
12. Ask a question only when a missing decision blocks correct implementation; otherwise use the blueprint defaults.
```

13.1 Reusable phase prompt template

```
Implement Phase <N>: <phase name> from the attached blueprint.

Before editing:
- inspect the current repository and completed phases;
- verify prerequisites and identify any conflict with the blueprint.

Then implement the complete phase, including migrations/configuration/tests/documentation required by that phase. Do not implement later-phase features. Run all relevant builds and tests, fix issues introduced by your changes, and finish with the required handover format from Section 13.
```

13.2 First Antigravity prompt

```
Read the attached Customer Feedback Intelligence Platform blueprint completely. Start with Phase 0 and Phase 1 only: discovery/environment verification, monorepo skeleton, PostgreSQL+PGVector Docker Compose, Angular/Spring Boot/FastAPI scaffolds, health endpoints, configuration templates, and README run instructions.

Do not implement CSV ingestion or AI features yet. Inspect the machine/repository first, preserve anything useful, then implement and verify Phase 1. Run builds and health checks where possible. Finish with changed files, exact commands, test results, blockers, and the exact Phase 2 prompt.
```

13.3 Context recovery prompt

```
Read the blueprint and inspect the repository. Determine the last fully completed phase from code, migrations, tests, and README, not from assumptions. Report gaps briefly, repair only blockers in completed phases, then implement the next incomplete phase. Preserve the defined architecture and finish with the standard handover.
```

14. Final Acceptance Checklist

- ☐ Fresh setup instructions work from repository root.
- ☐ PostgreSQL starts with PGVector enabled.
- ☐ Angular, Spring Boot, and FastAPI health checks pass.
- ☐ CSV preview/mapping/validation works.
- ☐ Valid rows persist; invalid rows and duplicates are reported.
- ☐ Real-model analysis produces validated structured data.
- ☐ Embeddings are stored and semantic search works.
- ☐ Dashboard metrics agree with direct database calculations.
- ☐ Agent uses approved tools and returns evidence-backed answers.
- ☐ Failures, retries, prompt versions, and model names are traceable.
- ☐ No secrets or sensitive sample data exist in Git.
- ☐ Automated tests and one complete demo flow pass.

14.1 Demonstration scenario

8. Start all services and open the dashboard.
9. Upload a sample banking feedback CSV.
10. Show progress, one failed validation, and successful AI processing.
11. Open dashboard trends and filter by product/date.
12. Inspect a negative feedback record and its original text.
13. Run a semantic search such as “customers unable to access their accounts.”
14. Ask the agent why negative sentiment changed and request suggested investigations.
15. Open supporting feedback references and explain the deterministic-versus-agentic design.

14.2 Resume-ready description

Built an agentic customer-feedback intelligence platform using Angular, Spring Boot, Python FastAPI, LangGraph, PostgreSQL/PGVector, and a real LLM/embedding provider. Implemented traceable CSV ingestion, structured feedback classification, semantic search, evidence-grounded conversational analytics, trend analysis, and actionable recommendation workflows with validation, retry, and audit controls.

14.3 Interview explanation

“I intentionally did not make every step autonomous. Mandatory feedback processing is a deterministic LangGraph workflow because every record must be validated, analyzed, embedded, and persisted predictably. Open-ended manager questions use a supervisor agent that selects controlled metrics, trend, retrieval, and recommendation tools. Numerical claims come from SQL or code, while the LLM interprets verified evidence. This gives autonomy where it adds value without sacrificing reliability.”

15. Locked Decisions and Open Inputs

15.1 Locked decisions

- Initial domain: banking-style customer feedback.
- Initial ingestion: CSV with adapter-oriented design.
- Frontend: Angular.
- Business backend: Spring Boot.
- AI orchestration: Python FastAPI + LangGraph.
- Storage: PostgreSQL + PGVector.
- Architecture: one controlled processing graph plus one supervisor analytics graph.
- Final demo uses a real chat model and embedding model.
- Phase-wise incremental implementation.

15.2 Inputs to resolve before Phase 3

- Available model provider, endpoint, API version, chat deployment/model, embedding deployment/model.
- Expected embedding dimension from the selected model.
- Available quota/rate limits and cost constraints.
- Whether background processing will use an in-process worker initially or a simple durable job mechanism.
- Final chart library selection.

Default when not blocked Prefer the simplest production-minded option that satisfies the acceptance criteria. Record the decision in README/ADR and avoid introducing infrastructure that is not needed for Version 1.